Ja, das ist ein sehr kleiner Datensatz. Bei ~7 Merkmalen und 2000 Zeilen sind neuronale Netze meist **unnötig** und oft schlechter als einfachere Modelle.

Was ein „kleiner Datensatz“ in der Praxis bedeutet

Größe	Beispiel	Wann sinnvoll
Sehr klein	2–5 Tsd. Zeilen, einige bis ein Dutzend Merkmale	Fast immer klassisches ML
Klein	10–50 Tsd. Zeilen	Meist weiterhin besser: Bäume / Boosting
Mittel	ab 100 Tsd. Zeilen, komplexe Daten (Bilder, Text)	Neuronale Netze werden sinnvoll
Groß	Millionen Zeilen / große Modelle	Neuronale Netze (oder größere)

Deine 2000 Zeilen × 7 Merkmale sind ein **sehr kleiner** Datensatz.

Lohnen sich neuronale Netze?

Eher nicht. Bei dieser Größe:

- Neuronale Netze **überanpassen** sich leicht (Overfitting).
- Sie sind langsamer im Training und schwerer zu interpretieren.
- Einfachere Modelle liefern meist **bessere oder vergleichbare** Ergebnisse.

Bessere Optionen für den Start:

Modell	Warum es passt
Random Forest	Robust gegen Overfitting, wenig Tuning
XGBoost / LightGBM	Oft das beste Ergebnis bei Tabellendaten
Logistic Regression / Linear	Einfach, schnell, interpretierbar
SVM	Gut bei kleineren Datensätzen

Wann doch ein neuronales Netz?

- Wenn die Daten **nichtlinear und komplex** sind und klassische Modelle schwach abschneiden.
- Wenn du neuronale Netze **lernen** willst (dann gern, aber das Ergebnis wird nicht unbedingt besser).
- Wenn später ein deutlich größerer Datensatz geplant ist.

Zusammenfassung

- 2000 Zeilen + 7 Merkmale = **sehr kleine Daten**.
- Neuronale Netze kannst du verwenden, **musst du aber nicht** – und sie liefern wahrscheinlich kein besseres Ergebnis als XGBoost oder Random Forest.
- Auf so einem Datensatz trainierst du ein Modell problemlos auch auf einem schwachen Rechner (ohne GPU).

Soll ich ein einfaches Beispiel in Python zeigen (z. B. XGBoost oder ein kleines neuronales Netz) mit Daten aus einer CSV-Datei? Oder dir helfen zu entscheiden, welches Modell zu deiner Aufgabe passt (Klassifikation oder Regression)?

Wie viele Epochen soll ich ungefähr einstellen? Ich nutze Early Stopping.

Mit Early Stopping musst du die Zahl der Epochen nicht genau raten. Du setzt eher eine Obergrenze, und Early Stopping beendet das Training von selbst, sobald sich das Modell nicht mehr verbessert.

Empfehlung für deinen Fall (ca. 2000 Zeilen, 7 Merkmale)

Parameter	Vorgeschlagener Wert	Kommentar
Maximale Zahl der Epochen	**100–300**	Sichere Obergrenze
Patience (Early Stopping)	**10–20**	Wie viele Epochen ohne Verbesserung gewartet wird
Monitor	val_loss	Meist die beste Wahl

Beispiel-Einstellungen

- epochs=200
- patience=15
- min_delta=0.001 (optional – die kleinste Verbesserung, die zählt)

Warum so?

- Auf einem kleinen Datensatz konvergiert das Netz meist schnell (oft in 20–80 Epochen).
- Early Stopping bricht das Training ohnehin früher ab – eine hohe Grenze schadet also nicht.
- Eine zu niedrige Grenze (z. B. 20–30) kann das Training vor dem Optimum abbrechen.
- Eine zu hohe Grenze (z. B. 1000+) bringt nichts, weil Early Stopping ohnehin früher stoppt.

Weitere Tipps

- Immer einen Validierungsdatensatz verwenden (z. B. 15–20 % der Daten).
- Erreicht das Modell sehr schnell ein Plateau – kannst du die Lernrate senken.
- Achte bei so wenigen Daten darauf, dass sich das Modell nicht überanpasst (Regularisierung, Dropout, kleineres Netz).

Zusammenfassung: 200 Epochen mit **patience=15** einstellen. Early Stopping entscheidet selbst, wann Schluss ist.